Implementation Guide - AI-Powered E-Commerce DevOps & Cloud-Native Delivery Platform

This guide takes you from an empty folder to the complete platform described in the Davine Technologies architecture report, using **only free tools**. Every step has four parts:

- **What** you are building
- **Why** it exists in a real DevOps platform
- **Do it** - the exact commands
- **Check** - how you know it worked (and what to screenshot for your submission)

Work through the steps **in order**. Each one builds on the previous one.

Step 0 - The free-tier plan (read this first)

You have no paid cloud account, so each paid component of the report is replaced by a free equivalent that teaches exactly the same skills:

Report requirement	Paid-cloud version	What we use (free)
Cloud Kubernetes (EKS/AKS/GKE)	Managed cluster, ~$70+/month	**kind** - a real Kubernetes cluster running inside Docker on your computer
Cloud VM provisioned by Terraform	EC2	Terraform builds the kind cluster (`target=local`). The **same code** also contains an AWS EC2 module (`target=aws`) you can plan/apply later if you get credits
Container registry (ECR/ACR)	Paid storage	A **local registry** container (`registry:2`) - behaves exactly like a real registry
CI server	Hosted CI	**Jenkins in Docker** (free) + **GitHub Actions** (free for public repos)
Logging/monitoring SaaS	Datadog, etc.	Self-hosted **Prometheus, Grafana, ELK** in Docker/Helm
AI analysis	Paid LLM API	Built-in **rules engine** (offline, free) + optional free local LLM via **Ollama**
"Cloud deployment"	Paid	`terraform` `target=aws` on a free-credit account **or** Oracle Cloud "Always Free" VM + Ansible (Step 16)

Honest note on "cloud": the architecture report asks for a cloud deployment. Steps 1-15 prove the whole platform works locally. Step 16 shows the free routes to run it on a real internet-facing server. If your course accepts a local Kubernetes cluster as the "cloud-native environment", Step 16 is optional; if it insists on a public cloud, do Step 16 as well.

Computer requirements

- 8 GB RAM minimum (16 GB comfortable), 30 GB free disk, 4 CPU cores.
- **Windows:** install WSL2 (`wsl --install -d Ubuntu` in an admin PowerShell) and Docker Desktop with the "Use WSL 2 engine" option; do everything inside the Ubuntu terminal. **macOS:** use Docker Desktop and `brew` instead of `apt` in `scripts/setup.sh`. **Linux:** works as is.
- **Low-RAM trick:** do not run everything at once. Run one stack at a time (Step 11 monitoring, then stop it, then Step 12 ELK). The guide tells you when to stop things.
- No suitable computer? **GitHub Codespaces** gives free monthly hours of a cloud Linux machine with Docker (check GitHub's current free allowance).

Step 1 - Linux foundation

What: A Linux environment with the tools the project needs. **Why:** Linux is the operating system under Docker, Jenkins, Kubernetes and every cloud server. Every later step is a Linux command, so you must be comfortable with a small core set.

Do it

```bash
# 1. get the project
cd ~
# unzip the project here (after Step 2 you can `git clone` it instead)
cd ai-ecommerce-devops

# 2. install every tool (docker, kubectl, kind, helm, terraform, ansible, trivy, gitleaks
chmod +x scripts/*.sh security/*.sh
./scripts/setup.sh
newgrp docker            # make the docker group active in this terminal
```

Linux skills the report lists - practise each once (and screenshot):

Skill	Commands	Why it matters here
Package management	`apt update`, `apt install -y jq`, `apt list --installed | grep docker`	`setup.sh` and Ansible use apt
Users & groups	`id`, `sudo adduser demo`, `sudo usermod -aG docker demo`	the `docker` group = permission to use Docker; Ansible creates a `deploy` user
Permissions	`ls -l scripts/`, `chmod +x file`, `chmod 600 secret`, `chown user:group file`	scripts must be executable; secrets must not be world-readable
Processes	`ps aux | grep node`, `top` / `htop`, `kill <pid>`	find and stop runaway processes
Services	`systemctl status docker`, `sudo systemctl restart docker`, `journalctl -u docker -n 50`	Docker, Jenkins and SSH are services
Networking	`ss -tulpn` (what listens on which port), `curl -i localhost:8080`, `ping`, `ip a`	"is anything on port 8080?" is the most common debugging question
Disk/memory	`df -h`, `free -h`, `du -sh *`	ELK and Docker images eat disk and RAM
Env variables	`export FOO=bar`, `echo $FOO`, `env`	how Docker/K8s pass config to the app
SSH	`ssh-keygen -t ed25519`, `ssh user@host`	how Terraform/Ansible reach a cloud server

Check: `docker run --rm hello-world` prints "Hello from Docker!" and `kind version`, `helm version`, `terraform version`, `ansible --version` all print versions.

Step 2 - Git & GitHub (single source of truth)

What: Put the project in Git, publish to GitHub, use a branching model: `main` (always deployable) <- `develop` (integration) <- `feature/*` (one per change). **Why:** Everything - app code, Dockerfiles, Terraform, pipelines - is stored as text in one repository. CI/CD reacts to changes in it. That is the "single source of truth" in the report.

Do it

```bash
git config --global user.name  "Your Name"
git config --global user.email "you@example.com"

cd ai-ecommerce-devops
git init -b main
# generate the lockfile that makes builds reproducible (needs Node 18+)
( cd application/backend && npm install )
git add .
git status                                  # confirm no node_modules/.env/secrets are l
git commit -m "chore: initial platform scaffold"

# Create an EMPTY public repo named ai-ecommerce-devops on github.com (no README), then:
git remote add origin https://github.com/YOUR-USERNAME/ai-ecommerce-devops.git
git push -u origin main

# branching model
git checkout -b develop && git push -u origin develop
git checkout -b feature/add-readme-badge     # do work here
echo "" >> README.md && git commit -am "docs: tweak readme" && git push -u origin feature
```

On GitHub open a **Pull Request** `feature/add-readme-badge -> develop`, review it, merge. Later open a PR `develop -> main` for a "release". In Settings -> Branches add a protection rule for `main` ("Require a pull request before merging") - screenshot it.

Why each command: `git init` creates the history database; `git add/commit` records a snapshot; `remote/push` uploads it; branches isolate work; PRs are where code review and CI checks happen.

Check: GitHub shows 3 branches, one merged PR, and the Actions tab will start running `ci.yml` (Step 8).

Step 3 - The application (what we are delivering)

What: `application/` contains a small but real e-commerce app, because the report says the app already exists and the work is the platform around it. **If your course gave you its own app, put it in `application/` instead and adapt the Dockerfile** (the platform files stay the same).

- `application/backend` - Node.js/Express REST API: `GET /api/products`, `POST /api/orders`, plus operational endpoints `/health` (liveness), `/ready` (readiness, checks the DB), `/metrics` (Prometheus), JSON logs.

- `application/frontend` - static HTML/JS storefront.
- PostgreSQL stores products/orders. If `DATABASE_URL` is unset the backend uses an in-memory store (handy for tests).

This is the **application analysis** from the report's scope: ports (3000 backend, 8080 frontend, 5432 DB), env vars (`DATABASE_URL`, `PORT`, `ENABLE_CHAOS`, `APP_VERSION`), health endpoints, logs (JSON to stdout).

Do it (optional quick run without Docker)

```bash
cd application/backend
npm install
npm test                        # unit tests of the API
npm start &                     # runs on :3000 with the in-memory store
curl localhost:3000/health
curl localhost:3000/api/products
kill %1
```

Step 4 - Bash automation (`scripts/`)

What: Five reusable scripts from the report plus two extras. **Why:** Anything you type twice should be a script - consistent, reviewable, and callable from Jenkins.

Script	Purpose	Key ideas to explain
`setup.sh`	install all tools	`set -euo pipefail` (stop on error/unset var/failed pipe), `have()` function = idempotent (re-runnable)
`deploy.sh`	`compose`, `local` (kind+Helm) or `down`	functions, `case`, tags images with the git commit hash
`health-check.sh`	polls `/health` and `/api/products` with retries; exit code 0/1	exit codes are how Jenkins knows pass/fail
`backup.sh`	`pg_dump` to a gzip file with retention (keeps newest 7)	pipes, `date`, `ls -t | tail -n +N | xargs rm`
`cleanup.sh`	prune Docker, delete old logs/backups, print disk/memory/load	system checks
`deploy-strategy.sh`	rolling / canary / blue-green / rollback	used in Step 14
`logs.sh`	tail logs or show only errors	`grep` on JSON logs

Do it

```bash
bash -n scripts/deploy.sh            # syntax check without running
./scripts/health-check.sh http://localhost:8080 || echo "nothing running yet - expected"
./scripts/cleanup.sh                 # safe to run any time
```

Step 5 - Docker & Docker Compose

What: Package the app as images and run frontend + backend + database together. **Why:** "Works on my machine" disappears when the app and its dependencies travel in an image.

5.1 The backend Dockerfile (`docker/Dockerfile`) line by line

```dockerfile
```

```
FROM node:20-alpine AS deps          # stage 1: build stage, small Alpine Linux base
WORKDIR /app
COPY application/backend/package*.json ./
RUN npm install --omit=dev ...       # install ONLY runtime dependencies

FROM node:20-alpine                  # stage 2: final image starts clean (multi-stage =
COPY --from=deps /app/node_modules ./node_modules   # take only the installed modules
COPY application/backend/src ./src   # then the code
USER node                            # run as an unprivileged user (least privilege)
HEALTHCHECK ... wget .../health      # Docker itself checks the app is alive
CMD ["node", "src/server.js"]        # the process the container runs
```

`docker/.dockerignore` keeps `node_modules`, `.git`, Terraform state, etc. **out of the build context** (faster builds, no secrets leaking into images). `docker/Dockerfile.frontend` uses the unprivileged Nginx image.

5.2 Nginx as web server + reverse proxy (`docker/nginx.conf`)

The browser talks only to Nginx on port 8080: static files are served directly; anything under `/api/` is forwarded to `http://backend:3000`. `backend` is a DNS name provided by Compose (and later by the Kubernetes Service of the same name). Nginx also writes access and error logs - those feed ELK in Step 12.

5.3 Docker Compose (`docker/docker-compose.yml`)

Three services on one private network: `db` (Postgres + named volume so data survives restarts + healthcheck), `backend` (waits for a healthy db via `depends_on: condition: service_healthy`), `frontend` (publishes 8080).

Do it

bash

```
docker compose -f docker/docker-compose.yml up -d --build
docker compose -f docker/docker-compose.yml ps          # all 'healthy'/'running'
./scripts/health-check.sh compose
# open http://localhost:8080 in the browser, add items, press Checkout
docker images | grep shop                               # your images
docker network ls | grep shop                           # the network
docker volume ls | grep shop                            # the DB volume
curl localhost:8080/api/products
docker compose -f docker/docker-compose.yml logs backend | tail
./scripts/backup.sh compose && ls backups/              # backup script
```

Check: the storefront loads, an order returns "Order #1 placed". Screenshot `docker compose ps`.

Leave Compose running - Steps 11 and 12 monitor it. (Stop it with `docker compose -f docker/docker-compose.yml down`.)

Step 6 - Kubernetes (kind) and Helm

What: Run the same containers on Kubernetes, first with plain YAML (to learn), then with Helm (how it is done in practice). **Why:** Kubernetes restarts failed containers, scales them, rolls out new versions without downtime, and gives service discovery, config and secrets management.

6.1 Concepts mapped to files in `kubernetes/`

Object	File	Meaning
Namespace	`namespace.yaml`	a folder/boundary named `shop` for all our objects
Deployment (creates Pods)	`deployment.yaml`	"keep 2 copies of this container running"; includes `RollingUpdate`, probes, resource limits, `securityContext`
Service	`service.yaml`	stable name + load balancing in front of the pods (`backend`, `frontend`)
ConfigMap	`configmap.yaml`	non-secret config as env vars
Secret	`secret.yaml`	DB password (base64 at rest; demo value only)
Ingress	`ingress.yaml`	HTTP entry from outside the cluster to the frontend Service
HPA	`hpa.yaml`	autoscale 2-6 pods at 70% CPU (needs metrics-server)
kind config	`kind-config.yaml`	1 control-plane + 2 workers; host port 8081 -> ingress

Probes: `livenessProbe` (/health) restarts a stuck container; `readinessProbe` (/ready) keeps a pod out of the Service until it can talk to the DB. **Rolling update:** `maxSurge: 1, maxUnavailable: 0` = start one new pod, wait until ready, only then stop an old one -> zero downtime.

6.2 Create the cluster, ingress and registry

Choose ONE way to create the cluster:

- **Script (simplest):** `./scripts/deploy.sh cluster` - starts the local registry, creates the 3-node kind cluster, tells every node how to reach the registry, and installs the ingress controller. **or**
- **Terraform (Step 9):** `terraform apply` creates the cluster; then run `./scripts/deploy.sh cluster` (it skips creation because the cluster exists, but still wires up the registry and ingress).

6.3 Learn with raw manifests

```bash
```

```
./scripts/deploy.sh cluster
./scripts/deploy.sh build latest         # builds + pushes localhost:5001/shop-backend:la

kubectl apply -f kubernetes/namespace.yaml
kubectl apply -f kubernetes/configmap.yaml -f kubernetes/secret.yaml -f kubernetes/postgr
kubectl apply -f kubernetes/deployment.yaml -f kubernetes/service.yaml -f kubernetes/ingr
kubectl -n shop get all
kubectl -n shop describe pod -l app=shop-backend | tail -30    # events = first place to
./scripts/health-check.sh k8s                                   # http://localhost:8081
```

Experiments (screenshot each):

```bash
kubectl -n shop scale deploy/shop-backend --replicas=4          # scaling
kubectl -n shop delete pod -l app=shop-backend --wait=false     # self-healing: pods come
kubectl -n shop logs deploy/shop-backend --tail=20
kubectl -n shop exec deploy/shop-frontend -- wget -qO- http://backend:3000/health   # ser
```

Before Helm: `kubectl delete -f kubernetes/deployment.yaml -f kubernetes/service.yaml -f kubernetes/ingress.yaml -f kubernetes/postgres.yaml -f kubernetes/secret.yaml -f kubernetes/configmap.yaml` (Helm refuses to take over objects it did not create).

6.4 Helm - package it as one unit (`helm/ecommerce/`)

`Chart.yaml` = metadata, `values.yaml` = every tunable setting, `templates/` = the YAML with `{{ .Values.x }}` placeholders. One command installs everything; one command upgrades; one command rolls back.

```bash
helm lint helm/ecommerce
helm template shop helm/ecommerce | less          # see the final YAML Helm would send
./scripts/deploy.sh local                          # builds, pushes, helm upgrade --inst
helm list -n shop
helm history shop -n shop                          # revisions = your rollback points
```

Step 7 - Jenkins CI/CD

What: A pipeline that turns a Git push into a verified deployment. **Why:** Humans forget steps; a pipeline never does. This is the heart of the report.

Pipeline (from `jenkins/Jenkinsfile`, mirrors the report diagram): `Checkout -> Install deps -> Build(lint) -> Test -> Security checks (gitleaks | npm audit | trivy config, in parallel) -> Docker build -> Image scan (trivy) -> Push image -> Deploy (helm) -> Health check` and on failure it automatically runs `helm rollback`.

Do it

```bash
# 1. cluster must exist (Step 6). Give Jenkins a kubeconfig that works from inside Docker
kind get kubeconfig --internal --name shop > jenkins/kubeconfig
# 2. start Jenkins (first build downloads tools: ~5-10 min)
docker compose -f jenkins/docker-compose.yml up -d --build
docker compose -f jenkins/docker-compose.yml exec jenkins cat /var/jenkins_home/secrets/i
```

3. Open http://localhost:8090, paste the password, choose **Install suggested plugins**, create an admin user.
4. **New Item -> Pipeline** named `ecommerce`. Under Pipeline: Definition = Pipeline script from SCM, SCM = Git, Repository URL = your GitHub URL (public repo needs no credentials), Branch = `*/main`, Script Path = `jenkins/Jenkinsfile`. Save.
5. **Build Now**. Watch Stage View - every stage turns green; Console Output shows the commands.

Why these choices:

- The Jenkins container mounts `/var/run/docker.sock`, so `docker build` runs on your host daemon (fast cache). This is a lab shortcut; in production use isolated build agents.
- `localhost:5001` works for both the host daemon (push) and the kind nodes (pull, via the containerd mapping `deploy.sh` sets up).
- `pollSCM('H/2 * * * *')` makes Jenkins check GitHub every ~2 minutes - no public URL needed.
- If the pipeline fails on `npm audit`, that is the security gate working: run `npm audit fix` in `application/backend`, commit, push.

Check: a green pipeline, then change a line in `application/frontend/index.html`, commit, push, and within ~2 minutes a new build deploys it - refresh http://localhost:8081. That is "GitHub push -> Jenkins -> Docker -> Kubernetes".

Step 8 - GitHub integration & GitHub Actions

- **Polling** (already configured) is enough. A faster **webhook** needs Jenkins reachable from the internet: run `ngrok http 8090` (free account) and add `https://<id>.ngrok-`

`free.app/github-webhook/` under GitHub Settings -> Webhooks, then enable "GitHub hook trigger" in the job.

- `.github/workflows/ci.yml` is the "CI alternative": on every push/PR GitHub runs tests, npm audit, gitleaks, a Docker build, a Trivy image scan, `helm lint`, and the AI-analyser tests. Open the repo's **Actions** tab and screenshot a green run. Together with a branch-protection rule requiring it, this blocks bad PRs from reaching `main`.

Step 9 - Terraform (Infrastructure as Code)

What: Declare infrastructure in `.tf` files; Terraform makes reality match. **Why:** Environments become repeatable and reviewable instead of hand-clicked.

Files (matches the report): `provider.tf` (which plugins), `variables.tf` (inputs), `main.tf` (what to create), `outputs.tf` (results), `modules/` (reusable blocks), `environments/*.tfvars` (per-environment values). `var.target` switches between `local` (the `kind-cluster` module) and `aws` (the `aws-ec2` module).

Do it - free local target

```bash
kind delete cluster --name shop 2>/dev/null || true     # let Terraform own the cluster
cd terraform
terraform init                                          # downloads providers
terraform fmt -check && terraform validate              # style + syntax
terraform plan  -var-file=environments/local.tfvars     # preview: "+ kind_cluster.this w
terraform apply -var-file=environments/local.tfvars     # type yes
terraform output
kubectl --kubeconfig ~/.kube/shop-config get nodes      # 3 nodes Ready
cd ..
./scripts/deploy.sh local                               # registry + ingress + app on the
```

Key ideas to explain: **state** (`terraform.tfstate` remembers what exists - never commit it), **plan before apply**, **idempotency** (running apply twice changes nothing), `terraform destroy` removes everything.

AWS target (only if you have credits; otherwise do `validate` + `plan` reading only): `terraform validate` works without credentials. To create real resources copy `environments/aws.tfvars.example` to `aws.tfvars`, set `admin_cidr` to your IP, `aws configure`, then `terraform apply -var-file=environments/aws.tfvars`. It creates an Ubuntu EC2 server, a key pair, a firewall (SSH only from you) and writes `ansible/inventory-aws.ini` for Step 10. **Always `terraform destroy` when done.**

Step 10 - Ansible (configuration management)

What: Playbooks that configure a server: packages, a `deploy` user, firewall, SSH hardening, Docker, and start the app. **Why:** Terraform creates the machine; Ansible configures what is inside it. Re-running changes nothing if already correct.

Structure: `inventory` (which machines), `group_vars/all.yml` (variables), `playbook.yml` (roles in order), `roles/common`, `roles/docker`, `roles/app` (each with `tasks/main.yml`).

Do it - free: a throw-away VM on your own computer Easiest is Multipass (free): `multipass launch 22.04 --name shop-vm --memory 2G --disk 15G`, then `multipass info shop-vm` for the IP, and write an inventory line `IP ansible_user=ubuntu ansible_ssh_private_key_file=~/.ssh/id_ed25519` (add your public key to the VM first). Or simply run against your own Ubuntu/WSL machine with the default `inventory` (`ansible_connection=local`).

```bash
cd ansible
ansible-playbook playbook.yml --syntax-check
ansible-playbook playbook.yml --check --diff -K -e deploy_application=false     # dry run
ansible-playbook playbook.yml -K -e deploy_application=false                     # apply (
ansible-playbook playbook.yml -K -e deploy_application=false                     # run aga
```

Set `-e repo_url=https://github.com/YOU/ai-ecommerce-devops.git` and drop `deploy_application=false` to also clone and start the app. Firewall and SSH hardening are applied only to remote hosts (never locks you out of your own laptop).

Step 11 - Monitoring: Prometheus + Grafana

What: Metrics (numbers over time) and dashboards. **Why:** Monitoring tells you that something is wrong (error rate up, pod down) before users complain.

How it works: the backend exposes `/metrics` (request latency histogram, order counter, CPU/memory). Prometheus scrapes it every 15 s and stores the series. Grafana queries Prometheus and draws panels. `alert-rules.yml` defines alerts (backend down, >5% 5xx, p95 latency >1 s).

11a Compose environment (lightest; Compose app from Step 5 must be running)

```bash
```

```
docker compose -f monitoring/docker-compose.monitoring.yml up -d
```

- Prometheus http://localhost:9090 -> Status -> Targets: `shop-backend` must be **UP**.
- Grafana http://localhost:3001 (admin/admin) -> Dashboards -> **Shop -> Application Overview**.
- Generate load for the graphs: `for i in $(seq 200); do curl -s localhost:8080/api/products >/dev/null; done`
- Trigger errors: `for i in $(seq 20); do curl -s localhost:8080/api/chaos/error >/dev/null; done` (the error-rate panel rises and `HighErrorRate` goes pending/firing in Prometheus Alerts).

11b Kubernetes environment

bash

```
helm repo add prometheus-community https://prometheus-community.github.io/helm-charts &&
helm upgrade --install kube-prom prometheus-community/kube-prometheus-stack -n monitoring
helm upgrade shop helm/ecommerce -n shop --reuse-values        # chart now creates a Serv
kubectl -n monitoring port-forward svc/kube-prom-grafana 3002:80    # Grafana: http://loc
```

Import our panels: Grafana -> Dashboards -> New -> Import -> upload `monitoring/grafana/dashboards/shop-overview.json`. The built-in "Kubernetes / Compute Resources" dashboards already show CPU, memory, pod and node status (the report's key metrics).

Stop it to free RAM: `docker compose -f monitoring/docker-compose.monitoring.yml down`, `helm uninstall kube-prom -n monitoring`.

Step 12 - Centralised logging: ELK

What: All container logs searchable in one place. **Why:** Metrics say something is wrong; logs say what happened.

Flow: containers write JSON lines to stdout -> Docker stores them as files -> **Filebeat** tails them and adds container info -> **Logstash** parses and names fields -> **Elasticsearch** indexes (`shop-logs-YYYY.MM.dd`) -> **Kibana** lets you search and chart.

Do it (stop monitoring first to save RAM; Compose app must be running)

bash

```
sudo sysctl -w vm.max_map_count=262144            # Elasticsearch requirement on Linux (WS
docker compose -f logging/elk/docker-compose.elk.yml up -d
docker compose -f logging/elk/docker-compose.elk.yml ps      # wait ~2 min until elastics
./logging/elk/setup-kibana.sh                                # creates the data view "sho
for i in $(seq 20); do curl -s localhost:8080/api/chaos/error >/dev/null; done
```

Open http://localhost:5601 -> Discover -> data view **Shop logs**. Search `level:error` or `app.level:error`; filter by `container.name`. Screenshot the errors. Note: this ELK stack collects the **Compose** deployment's logs. For the Kubernetes deployment use `kubectl logs` / `scripts/logs.sh errors k8s`.

Step 13 - DevSecOps

What: Security checks wired into the delivery process, not bolted on at the end.

Control	Tool	Where
Secret detection	**gitleaks**	Jenkins stage, GitHub Action, `security/run-scans.sh`
Dependency vulnerabilities	**npm audit**	same
Container image vulnerabilities	**Trivy** (`--severity CRITICAL --ignore-unfixed` blocks)	Jenkins "Image Scan", GitHub Action
Config/IaC misconfiguration	**Trivy config**	Jenkins (advisory)
Secure configuration	non-root user, `readOnlyRootFilesystem`, dropped Linux capabilities, `allowPrivilegeEscalation: false`, IMDSv2, encrypted EBS	Dockerfile, Helm, Terraform
Kubernetes Secrets	`shop-secret` (never in Git for real values)	`kubernetes/secret.yaml`, Helm `--set postgres.password=...`
Least privilege	`security/network-policy.yaml` (default deny, then only frontend->backend->db)	needs a policy-enforcing CNI to be enforced
Linux hardening	`security/linux-hardening.sh` (updates, ufw, fail2ban, sysctl)	run `sudo ./security/linux-hardening.sh --dry-run` first

Do it

```bash
docker build -f docker/Dockerfile -t shop-backend:local .
./security/run-scans.sh local
```

Prove the gate works (great for the demo): create a file with a fake AWS key, e.g. `echo 'AWS_SECRET_ACCESS_KEY="AKIAIOSFODNN7EXAMPLEWJalrXUtnFEMI/K7MDENG/bPxRfiCYEXAMPLEKEY"' > application/leak.txt`, `git add` + `git commit`, run `gitleaks detect --redact` - it fails. Then delete the file **and rewrite history or delete the commit** (`git reset --hard HEAD~1`) - removing a secret in a later commit does not remove it from history. Never use real keys for this test.

Step 14 - Deployment strategies

Strategy	Command	What happens	Best for
Rolling (default)	`./scripts/deploy-strategy.sh rolling <tag>`	pods replaced one at a time, zero downtime	everyday releases
Canary	`./scripts/deploy-strategy.sh canary <tag>` then `promote <tag>`	1 pod of the new version beside 2 old pods (~1/3 of traffic) - watch Grafana/Kibana, then promote or `rollback`	risky changes
Blue-Green	`./scripts/deploy-strategy.sh bluegreen <tag>`	full second copy ("green") started and tested; the Service selector flips instantly; flip back = instant rollback	zero-risk cut-over
Rollback	`./scripts/deploy-strategy.sh rollback`	Helm returns to the previous revision	any failure

Do it

bash

```
# Build and push a "v2" and "v3" of BOTH images (the version is baked into the backend as
# so you can tell versions apart with /api/version even without changing any code).
./scripts/deploy.sh build v2
./scripts/deploy.sh build v3
./scripts/deploy-strategy.sh canary v2
kubectl -n shop get pods -L track,color
for i in $(seq 30); do curl -s localhost:8081/api/version; echo; done | sort | uniq -c
./scripts/deploy-strategy.sh promote v2              # everyone on v2
./scripts/deploy-strategy.sh bluegreen v3            # green (v3) started, tested, then
curl -s localhost:8081/api/version                   # now v3
./scripts/deploy-strategy.sh rollback                # back to the previous Helm revisio
```

In a rolling update watch it live in a second terminal: `kubectl -n shop get pods -w`.

Step 15 - AI-assisted incident analysis

What: `ai/incident-analysis/analyze.py` reads error logs, groups them, and outputs the four things in the report: **error classification, severity, possible root cause, suggested**

remediation. Why: During an incident engineers lose time reading thousands of lines. The tool compresses them into a prioritised summary.

Three engines (all free):

1. `rules` (default) - an offline knowledge base of patterns (timeouts, auth, OOM, disk, schema, image pull...) with severity escalation when an error repeats 10+ times. Deterministic, so it is testable and demo-safe.
2. `ollama` - a local LLM: install from ollama.com, `ollama pull llama3.2`, then `--engine ollama`. Nothing leaves your machine.
3. `anthropic` - optional paid API if you export `ANTHROPIC_API_KEY`. If an LLM is unreachable the tool **falls back to rules**, so it never fails when you need it most.

Do it

```bash
cd ai/incident-analysis
python3 -m unittest discover -s tests -v                    # 7 tests
python3 analyze.py --file sample_logs/db_timeout.log        # offline demo
cd ../..
# live: with Compose + ELK running, create errors then analyse Elasticsearch directly
for i in $(seq 15); do curl -s localhost:8080/api/chaos/error >/dev/null; done
python3 ai/incident-analysis/analyze.py --elastic http://localhost:9200 --minutes 15
docker logs shop-backend-1 2>&1 | python3 ai/incident-analysis/analyze.py --stdin    # wi
```

Exit code is `1` when an incident is found (usable in cron/CI), `0` when healthy.

Step 16 - Cloud deployment, free options

The report's last layer: GitHub -> Jenkins -> Docker -> Registry -> Cloud -> Kubernetes -> Monitoring/Logging/Security.

Option A - Terraform on a cloud with free credits (AWS/Azure/GCP). New accounts typically receive trial credits or a free tier; **check the provider's current terms** (they change) and set a budget alert. Use Terraform `target=aws`, then Ansible with `inventory-aws.ini` (Step 9-10). The EC2 module + Ansible deploy the app with Docker Compose (a 1 GB free instance is too small for a Kubernetes+ELK stack; use `t3.small` if credits allow).

Option B - Oracle Cloud "Always Free" VM. Oracle offers always-free Arm VMs with several GB RAM (needs a card for identity verification, not charged if you stay inside Always Free - verify current limits). Create an Ubuntu VM in the console, add your SSH key, then:

```bash
# ansible/inventory-oracle.ini
[shop_servers]
<PUBLIC_IP> ansible_user=ubuntu ansible_ssh_private_key_file=~/.ssh/id_ed25519
ansible-playbook -i inventory-oracle.ini playbook.yml -e repo_url=https://github.com/YOU/
```

Open ports 80/8080 in the VM's security list. Install k3s (`curl -sfL https://get.k3s.io | sh -`) on the same VM if you want Kubernetes in the cloud, copy its kubeconfig, and point Helm at it.

Option C - Present the local cluster as the cloud-native environment. Everything Terraform/Helm/Kubernetes does on kind is identical on EKS/AKS/GKE; only the `target` module changes. State this clearly in your report and show the AWS Terraform module with `terraform validate`.

Step 17 - The end-to-end demo (10 minutes)

1. `git push` a visible change to `develop`, merge a PR to `main` (Step 2).
2. Jenkins picks it up; show the Stage View with security + scan stages (Step 7).
3. Browser: http://localhost:8081 shows the new version; `helm history shop -n shop` shows the revision (Step 6).
4. Grafana: traffic graphs rising while you refresh the page (Step 11).
5. Break it on purpose: hit `/api/chaos/error` (needs `ENABLE_CHAOS=true`: `helm upgrade shop helm/ecommerce -n shop --reuse-values --set backend.enableChaos=true`).
6. Show the alert/error-rate panel, then Kibana errors (Step 12), then the AI analysis output (Step 15).
7. Recover: `./scripts/deploy-strategy.sh rollback`, then `./scripts/health-check.sh k8s`.

Step 18 - Teardown (free your RAM and disk)

```bash
./scripts/deploy.sh down
docker compose -f jenkins/docker-compose.yml down -v
docker compose -f monitoring/docker-compose.monitoring.yml down -v
docker compose -f logging/elk/docker-compose.elk.yml down -v
cd terraform && terraform destroy -var-file=environments/local.tfvars   # if Terraform ma
./scripts/cleanup.sh --deep
```

Appendix - Report requirement checklist

Report deliverable	Where
GitHub repo, branching, PR workflow, docs	Step 2, `README.md`, `docs/`
Linux env, Bash automation, server configuration	Steps 1, 4, 10
Dockerfile, Docker Compose, container image	Step 5 `docker/`
Jenkins pipeline/Jenkinsfile, GitHub integration, automated deployment	Steps 7-8 `jenkins/`, `.github/`
Terraform, modules, Ansible playbooks	Steps 9-10 `terraform/`, `ansible/`
Namespace, Deployment, Service, ConfigMap, Secret, Ingress, Rolling deployment	Step 6 `kubernetes/`, `helm/`
Prometheus, Grafana dashboard	Step 11 `monitoring/`
Centralized logging, Kibana analysis	Step 12 `logging/`
Dependency scanning, image scanning, secret detection, hardening	Step 13 `security/`
AI-assisted log and incident analysis	Step 15 `ai/`
Architecture diagram, implementation/deployment/troubleshooting docs, README	`docs/`, `README.md`